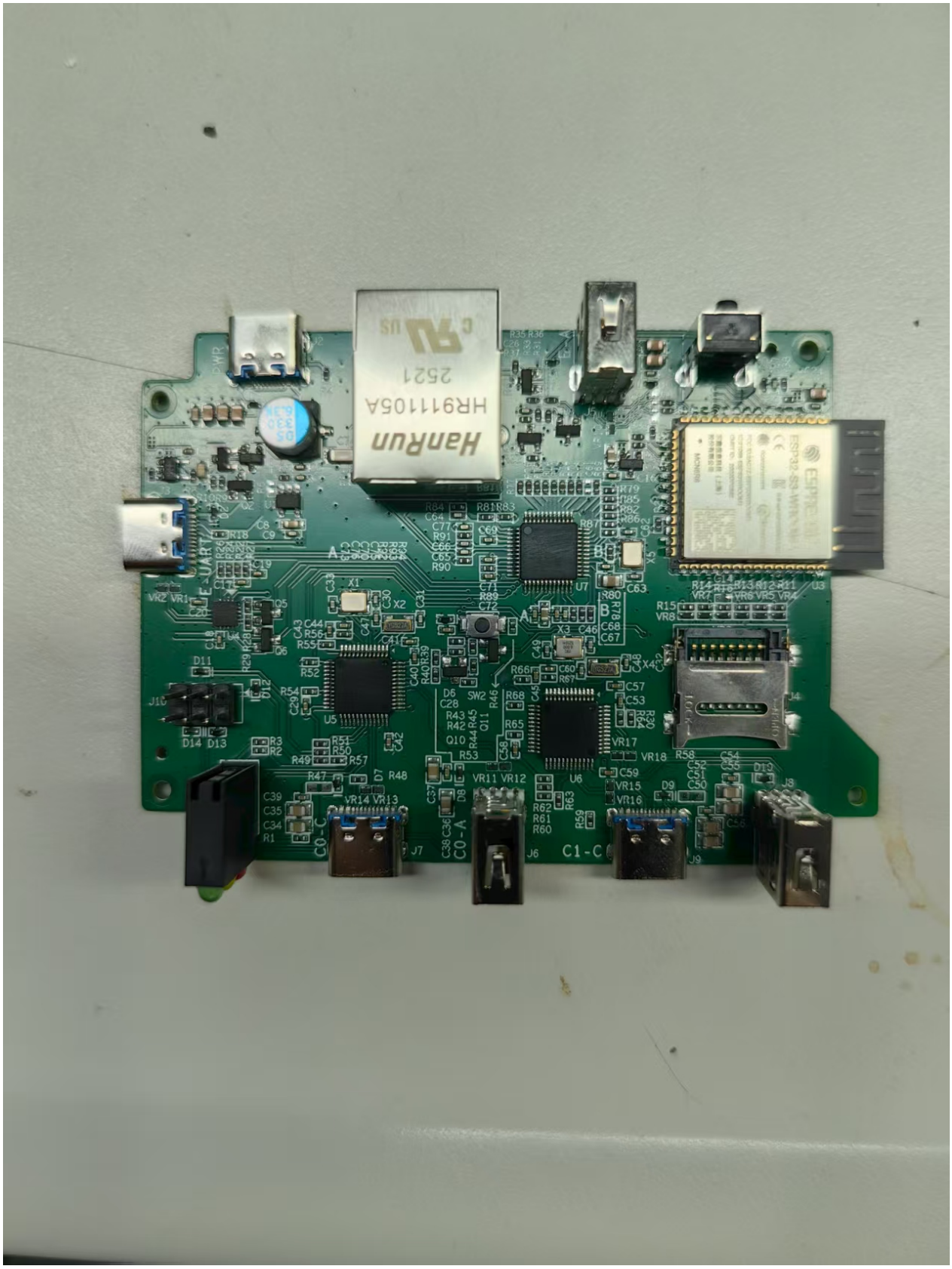
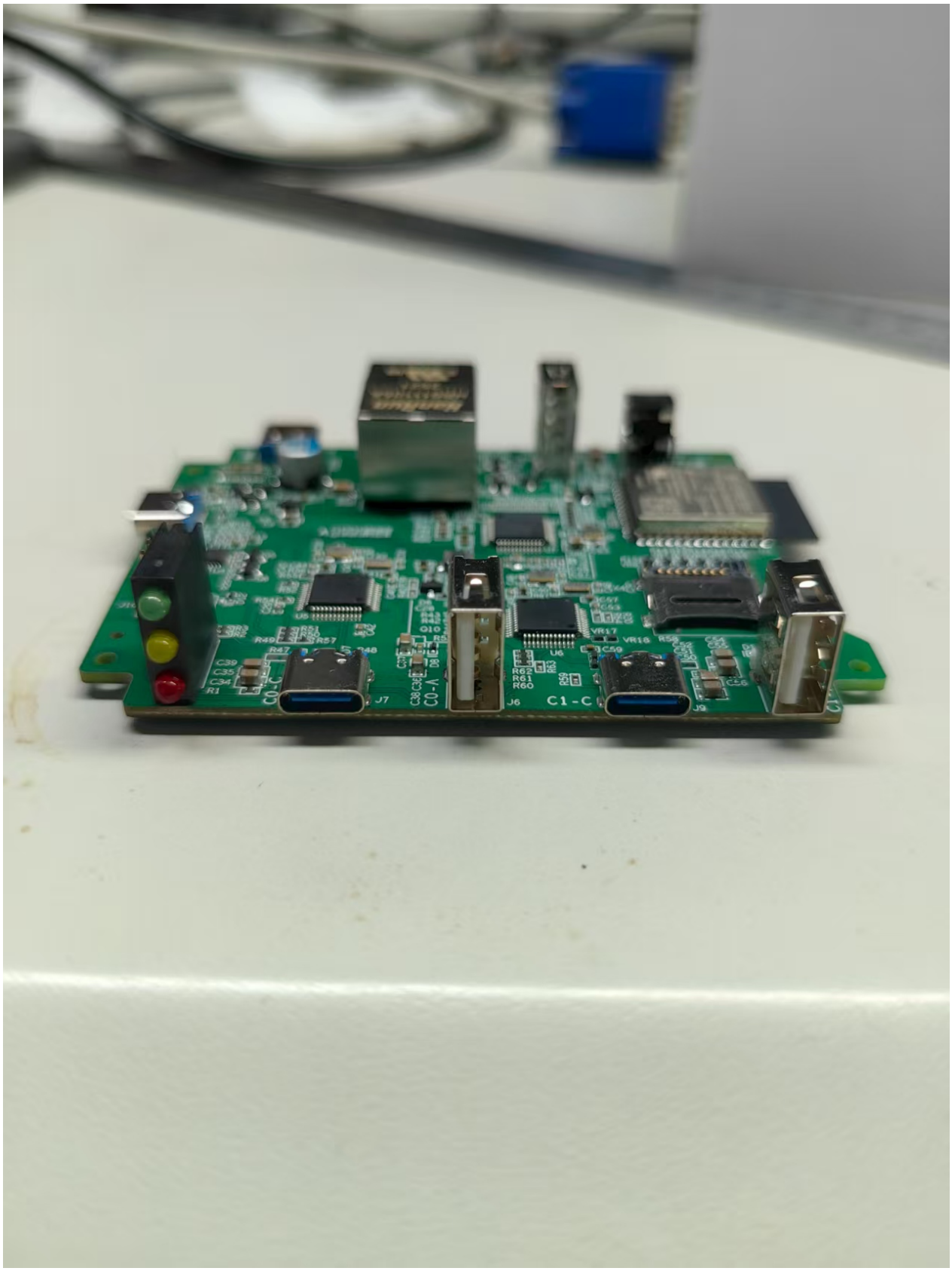
鼠标/键盘操作记录器



硬件的环境(当前的版本的形态)

1. 板子一个

2. TF 卡: 用来记录操作行为的. 记录的数据量,主要依赖卡的大小.
3. 下侧(鼠标键盘连接侧): 2 组 (usb-type-c, usb-host)+, 顺序是左侧一组是1号, 右边一组是2号, 3色LED灯
4. 左侧(命令控制侧): usb type-c 控制口.
5. 上侧(电源): USB-type-C 电源, LAN, usb-host, 2个按键: 上面一个是应用按键, 下面一个是RST按键.



鼠标键盘侧说明

1. 3个 LED 灯: 最上绿色: 代表1# 设备就绪. 中间黄色代表 2#设备就绪.

> 推荐1# 接鼠标, 2# 接键盘, 未来可能会根据鼠标,键盘不同的特性,进行内部优化.

需要的设备

1. HP M10 的鼠标.
   或
2. 其他的USB普通USB 鼠标. (应该也可以.)
3. 3 条usb-c 的数据线. (1条通信控制, 2条 连接鼠标,键盘.)

需要的软件

1. 主要就是PC上的串口工具: 用来在串口实现功能的控制.

连接说明

1. 2条 USB TYPE-C 的线, 分别接到鼠标键盘侧的 2个usb-c, 占用2个pc的usb口.
2. 1条 USB TYPE-C 的线, 是在照片左边接口, 和pc 连接.

这3条线最终都是和PC的USB连接的. (所以设备需要占3个PC的USB口.)

正常连接后, PC就会多一个USB 鼠标, 键盘可以直接现在的键盘.

软件概述

鼠标/键盘记录工具的使用, 是通过CLI 的方式来进行控制的. 设备会记录鼠标在记录时间内发生的所有事件. 回放的时候,就是对记录事件进行回放, 一个关键的注意点是

鼠标的起始位置 是无法记录的, 因为鼠标事件都是相对信息, 在启动时候必须要人工把鼠标放置到一个确定的起始位置.

主要的命令

1. alog: 启动鼠标记录. 如果没有文件名参数,系统会自动用时间作为参数.
2. astop: 停止鼠标记录.
3. aplay: 启动回放. 如果没有参数,就是启动最新录制的. 在aplay的时候,会留出5s的时间, 给用户重新设置鼠标的初始位置.

由于是2组usb通路, 所以启动,停止, 播放,都要带一个 --port=1|2 的参数, 分别对应 1#, 2# 的设备. (过时)

> v20250902 版本, 2个通道的数据写入一个日志文件. 播放就是针对设备, 2个通达的操作会同步执行.

一个关键的注意点是: 起始鼠标位置.

推荐使用方式

1. 系统有2个鼠标: 一个是用户自己操作的鼠标.(主鼠标) 一个是和设备连接的鼠标 (记录鼠标).
2. 用alog启动记录, 然后用主鼠标移动到目标的初始位置, 然后操作记录鼠标, 特别注意这个目标位置要自己能标记(明确知道在哪里). 可以用某个位置不改变的地方作为起始位置.
3. 当记录鼠标完成操作后. 用主鼠标激活串口工具窗口, 输入astop. 这个时候 就退出记录状态.同时显示了记录的文件名字的全路径.
4. 回放方式就是: aplay. 无参数就是最新的记录的文件. 命令输完后, 有5s的时间, 必须要把鼠标移动到第一次play的目标地址, 这样才能保持后续回防和第一次操作是基本一致的. 这个时候的移动,主鼠标,和记录鼠标都可以,移动到目标初始位置.

主鼠标 移动是不会被记录的, **记录鼠标**记录是会被记录的, 记录启动时候,用主鼠标移动到初始位置,是关键的一个过程.(启动时候不带延迟,记录开始了.) 回放时候, 有5s等待, 这个时候主鼠标,控制鼠标都可以用来移动到目标位置.

同样也有 **主键盘** 和**记录键盘**的概念, 如果只用一个键盘, 也是可以的, 那就是命令发出,通过程序发出, 也就不会影响正常逻辑.

> 回放的第一次: 有可能出现起始位置失控的问题. 这个是因为内部支持多类型的mouse, 有不稳定的情况. 这个只会出现一次. 后面再回放就不会出现这个问题了.

系统启动后,(如果mouse 已经插入了), 记录鼠标默认进入记录状态. 这个时候记录的时间已经启动. 如果真正的操作要过很长时间才开始,就不要使用这次几轮,因为如果启动回放, 这个很长时间的等待也被迫执行了. 退出记录,就是输入astop. 这个时候,系统不在记录鼠标操作事件.

推荐流程

0. 启动后, 就输出astop, 取消默认记录. (这个未来可以修改,不默认启动记录. 这个看实际操作情况,有时后默认启动记录会方便点. 不清楚windows首次发现鼠标是否,会在默认位置.)
1. 准备工作做好了以后, 使用alog xxxx.log 启动记录 (文件名自己定义), 用主鼠标移动到明确的起始位置, 然后开始使用记录鼠标.
2. 操作完毕后,用主鼠标 激活打串口窗口,输入astop 停止记录.
3. 回放使用: aplay xxxx.log , 然后在5s 内, 把鼠标移动到记录时候的起始位置.

特别注意:

1. 鼠标移动的速度, 可能太快会导致回放不准, 但是这个到底是多少快, 多少不准, 目前没有测量过.

软件使用

连接都成功后, PC 会出现 串口设备, 看对应的串口号.

1. 用串口工具, 打开这个串口, 设置 115200, 没有硬件流控, DTS, RTS 这些都不要选择.
2. 按板子的RST 按键,在板子的背后有2个按键的下面一个. (重新启动程序时候,可以按一下.)

> windows的串口工具,可能无法显示颜色信息,导致会有些控制字符显示和我下面的日志有点差异.

这样串口就会有如下的输出. (看到这个,就是说明软件进入正常状态了.)

```
ESP-ROM:esp32s3-20210327
Build:Mar 27 2021
rst:0x1 (POWERON),boot:0xb (SPI_FAST_FLASH_BOOT)
SPIWP:0xee
mode:DIO, clock div:1
load:0x3fce2810,len:0x10e4
load:0x403c8700,len:0x4
load:0x403c8704,len:0xb30
load:0x403cb700,len:0x2cfc
entry 0x403c88a8
I (264) octal_psram: vendor id    : 0x0d (AP)
I (264) octal_psram: dev id       : 0x02 (generation 3)
I (264) octal_psram: density      : 0x03 (64 Mbit)
I (266) octal_psram: good-die     : 0x01 (Pass)
I (270) octal_psram: Latency      : 0x01 (Fixed)
I (274) octal_psram: VCC          : 0x01 (3V)
I (278) octal_psram: SRF          : 0x01 (Fast Refresh)
I (283) octal_psram: BurstType    : 0x01 (Hybrid Wrap)
I (288) octal_psram: BurstLen     : 0x01 (32 Byte)
I (293) octal_psram: Readlatency  : 0x02 (10 cycles@Fixed)
I (298) octal_psram: DriveStrength: 0x00 (1/1)
I (303) MSPI Timing: PSRAM timing tuning index: 5
I (307) esp_psram: Found 8MB PSRAM device
I (310) esp_psram: Speed: 80MHz
I (313) cpu_start: Multicore app
I (749) esp_psram: SPI SRAM memory test OK
I (758) cpu_start: Pro cpu start user code
I (758) cpu_start: cpu freq: 160000000 Hz
I (758) app_init: Application information:
I (758) app_init: Project name:     mousekeypadplayer
I (763) app_init: App version:      62c7dcf-dirty
I (767) app_init: Compile time:     Aug 27 2025 10:16:58
I (772) app_init: ELF file SHA256:  000f8b41d...
I (776) app_init: ESP-IDF:          v5.4.2
```

```
I (780) efuse_init: Min chip rev:     v0.0
I (784) efuse_init: Max chip rev:     v0.99
I (788) efuse_init: Chip rev:         v0.2
I (792) heap_init: Initializing. RAM available for dynamic allocation:
I (798) heap_init: At 3FCAA790 len 0003EF80 (251 KiB): RAM
I (803) heap_init: At 3FCE9710 len 00005724 (21 KiB): RAM
I (809) heap_init: At 3FCF0000 len 00008000 (32 KiB): DRAM
I (814) heap_init: At 600FE088 len 00001F60 (7 KiB): RTCRAM
I (819) esp_psram: Adding pool of 8192K of PSRAM memory to heap allocator
I (826) spi_flash: detected chip: generic
I (829) spi_flash: flash io: dio
I (833) sleep_gpio: Configure to isolate all GPIO pins in sleep state
I (839) sleep_gpio: Enable automatic switching of GPIO sleep configuration
I (846) main_task: Started on CPU0
I (870) esp_psram: Reserving pool of 32K of internal memory for DMA/internal allocations
I (870) main_task: Calling app_main()
I (878) cli: Command history disabled
I (886) gpio: GPIO[41]| InputEn: 0| OutputEn: 0| OpenDrain: 0| Pullup: 1| Pulldown: 0| Intr:0
I (886) gpio: GPIO[42]| InputEn: 0| OutputEn: 0| OpenDrain: 0| Pullup: 1| Pulldown: 0| Intr:0
I (894) gpio: GPIO[40]| InputEn: 0| OutputEn: 0| OpenDrain: 0| Pullup: 1| Pulldown: 0| Intr:0
I (902) gpio: GPIO[39]| InputEn: 0| OutputEn: 0| OpenDrain: 0| Pullup: 1| Pulldown: 0| Intr:0
I (910) gpio: GPIO[1]| InputEn: 0| OutputEn: 0| OpenDrain: 0| Pullup: 1| Pulldown: 0| Intr:0
I (918) gpio: GPIO[2]| InputEn: 0| OutputEn: 1| OpenDrain: 0| Pullup: 0| Pulldown: 0| Intr:0
I (950) gpio: GPIO[2]| InputEn: 0| OutputEn: 0| OpenDrain: 0| Pullup: 1| Pulldown: 0| Intr:0
Name: asdfg
Type: SDHC
Speed: 40.00 MHz (limit: 52.00 MHz)
Size: 3840MB
CSD: ver=2, sector_size=512, capacity=7864320 read_bl_len=9
SSR: bus_width=4
I (966) pp: pp rom version: e7ae62f
I (966) net80211: net80211 rom version: e7ae62f
I (974) wifi_init: rx ba win: 6
I (974) wifi_init: accept mbox: 6
I (974) wifi_init: tcpip mbox: 32
I (974) wifi_init: udp mbox: 6
I (982) wifi_init: tcp mbox: 6
I (982) wifi_init: tcp tx win: 23040
I (982) wifi_init: tcp rx win: 23040
I (990) wifi_init: tcp mss: 1440
I (990) wifi_init: WiFi IRAM OP enabled
I (998) wifi_init: WiFi RX IRAM OP enabled
I (998) phy_init: phy_version 701,f4f1da3a,Mar  3 2025,15:50:10
I (1038) phy_init: Saving new calibration data due to checksum failure or outdated calibration data, mode(0)
I (1054) uart: queue free spaces: 8
I (1054) dcat: Initializing Concat Structure
I (1054) dcat: Concat Structure Initialized, State: IDLE
I (1054) d2f: uart task port 1 start

I (1054) uart: queue free spaces: 8
I (1062) dcat: Initializing Concat Structure
I (1062) dcat: Concat Structure Initialized, State: IDLE
I (1070) d2f: uart task port 2 start

I (1070) d2f: uart rx break
I (1070) sntp: Initializing and starting SNTP
I (1094) esp_eth.netif.netif_glue: 52:78:7d:16:87:b4
I (1094) esp_eth.netif.netif_glue: ethernet attached to netif
I (1102) eth_br: Ethernet Started
I (1102) alog: Directory /eMMC/applog exists
I (1302) cli: wifi not active
I (1302) cli: App Version: v20250826

Type 'help' to get the list of commands.
Use UP/DOWN arrows to navigate through command history.
Press TAB when typing command name to auto-complete.
I (1398) d2f: port 1: 1->15
I (1398) d2f: port 2: 1->15
I (1398) d2f: $VER:0.9.9:UNK$  <<--- 1# 没接设备
I (1398) d2f: $VER:0.9.9:HID$  <<----2# 接了设备.
I (1398) cmres: cx: 0.9.9, esp32: v20250826  <<=== 目前的版本
I (1398) d2f: Hid USB
I (1414) alog: 2# logfile /eMMC/applog/1970-01-01-08-00-00_2.log  <<== 2# 默认启动记录
I (1390) main_task: Returned from app_main()
```

- astop: 停止记录. (v20250902, 不需要参数了. 直接停止. 参数功能还保留,但是已经没有功能.)

下面是没有移动过记录鼠标的情况

```
I (1797) main_task: Returned from app_main()
cli> astop   <<=== 没有参数声明停止哪个端口的的记录.
astop: missing option --port=<p>
Command returned non-zero error code: 0x1 (ERROR)
cli> astop --port 0  <<==错误的端口, 端口只有 1, 2
invalid port Port is only 1 | 2
Command returned non-zero error code: 0x1 (ERROR)
cli> astop --port 1  <<=== 1# 没有设备,停止也没特别的功能.
cli> astop --port 2  <<=== 正确停止 2# 设备.
I (113241) alog: 2# logfile /eMMC/applog/1970-01-01-08-00-00_2.log, 0  <<== 没有移动鼠标,所以数据是0
```

v20250902以后, 只要astop

```
I (1419) main_task: Returned from app_main()
cli> astop
cli>
```

任何的命令都要给出具体的通道编号, 编号和具体的物理连接设备对应. 建议是 1# 接鼠标, 2# 接键盘. (过时)

- 停止记录时候的鼠标输出

```
I (34378) d2f: 2#: 2->5
I (34386) alog: 2# stopped
I (34386) d2f: 2#: 2->5
I (34394) alog: 2# stopped
I (34394) d2f: 2#: 2->5
I (34402) alog: 2# stopped
I (34402) d2f: 2#: 2->5
I (34410) alog: 2# stopped
I (34410) d2f: 2#: 2->5
I (34418) alog: 2# stopped
```

说明2#在停止的状态,

- 启动记录, 停止记录 (v20250902后, 不需要--port)

```
cli> alog random-move   <<===注意不要有后缀. , 文件名会自动增加 _$port.log
alog: missing option --port=<p>
Command returned non-zero error code: 0x1 (ERROR)
cli>cli> alog --port 2  random-move
I (36241) alog: 2# logfile /eMMC/applog/random-move_2.log
cli> I (3137765) d2f: start:2->5
I (97993) d2f: 2#: 2->5
I (98001) d2f: 2#: 2->5
I (98009) d2f: 2#: 2->5

...
I (101849) d2f: 2#: 2->5
I (101857) d2f: 2#: 2->5
I (101897) d2f: 2#: 2->5
cli>  astop --port 2
I (128513) alog: 2# logfile /eMMC/applog/random-move_2.log, 102 <<=== 102 条记录
```

v20250902 以后的版本

```
cli>
cli> alog random-move
I (132419) alog: logfile /eMMC/applog/random-move.log
cli> I (136763) d2f: 1#: 2->5
I (136771) d2f: 1#: 2->5
I (136779) d2f: 1#: 2->5
...
I (139947) d2f: 1#: 2->5
I (139955) d2f: 1#: 2->5
I (140035) d2f: 1#: 2->5
cli> astop
I (142291) alog: logfile /eMMC/applog/random-move.log, 378
```

- 回放记录 (v20250902后, 不需要--port)

必须路径参数, 现在没有最后的的记录说法.

```
cli> aplay --port 2
aplay: missing option <path>
Command returned non-zero error code: 0x1 (ERROR)
cli> aplay --port 2 random-move
I (270217) log2file: Move mouse to target postion, within 5s
I (275217) alog: 2# logfile /eMMC/applog/random-move_2.log
I (279193) alog: finished rec: 102    <<=== 回放102条
I (279193) log2file: Replay Finished
```

> 最新版本增加 --delay 的参数, 默认不给就是5s, 如果是设置 --delay 0 就是立刻回放, 不等待.

v20250902以后

```
cli> aplay random-move
I (213955) alog: logfile /eMMC/applog/random-move.log
cli> I (214459) alog: #1 unitsize 5
I (214659) alog: Move mouse to target postion, within 5s
I (216675) d2f: 1#: 2->5
I (216683) d2f: 1#: 2->5
I (216691) d2f: 1#: 2->5
I (216699) d2f: 1#: 2->5
I (216707) d2f: 1#: 2->5
I (216715) d2f: 1#: 2->5
I (216723) d2f: 1#: 2->5
...
I (217747) d2f: 1#: 2->5
I (217955) d2f: 1#: 2->5
I (222867) alog: finished rec: 378
```

- 取消回放 (v20250902后, 不需要--port)

在回放的过程中, 可以执行取消, 这样回放就终止了.

```
cli> acancel --port 2
```

v20250902以后

```
cli> I (339643) alog: #1 unitsize 5
I (339843) alog: Move mouse to target postion, within 5s
cli> acancel
I (345323) alog: canceled rec: 48
```

完整的执行例子

```
cli> aplay --port 2 circle
I (54370) alog: 2# logfile /eMMC/applog/circle_2.log
I (54370) alog: Move mouse to target postion, within 5s
I (70650) alog: finished rec: 953   <<---正常完成是 953条.

cli> aplay --port 2 circle --delay 2  <<--增加等待参数
I (88994) alog: 2# logfile /eMMC/applog/circle_2.log
I (88994) alog: Move mouse to target postion, within 2s
cli> acancel --port 2  <<===执行取消.
I (95970) alog: canceled rec: 466   <<---没有播放完毕, 停止了.
cli>
```

v20250910以后, 增加日志解析和增加鼠标接口控制.

- dump 日志, 增加日志解析.

m10的鼠标说明:

第一个数值: 是前后执行间隔的时间, 后面5个bytes 是m10的通信协议. byte1: button (5bit有效), byte2~4, x,y 的12bit 的组合编码. wheel 滚轮.

M10: x, y的范围 -2048~2047, wheel -128~127
button: 低5bit, 最低是 左按键.

下面的日志是启动录制,然后按了鼠标左按键, 然后右按键,又稍微移动了.

```
cli> alog mouseevt  <<===做一个日志分析
I (1124303) alog: logfile /eMMC/applog/mouseevt.log
...
cli> alog mouseevt
I (1124303) alog: logfile /eMMC/applog/mouseevt.log
cli> adump --port 1 mouseevt
I (1163279) alog: logfile /eMMC/applog/mouseevt.log
I (1163287) log2file: numrec 30, recsize 9
I (1163287) log2file: 0 01 00 00 00 00 |b:1 x:0 y:0 w:0     <<==左按键
I (1163295) log2file: 127 00 00 00 00 00 |b:0 x:0 y:0 w:0
I (1163295) log2file: 1384 02 00 00 00 00 |b:2 x:0 y:0 w:0  <<==右
I (1163303) log2file: 143 00 00 00 00 00 |b:0 x:0 y:0 w:0
I (1163303) log2file: 3936 00 01 20 00 00 |b:0 x:1 y:2 w:0
I (1163319) log2file: 7 00 01 00 00 00 |b:0 x:1 y:0 w:0
I (1163319) log2file: 7 00 03 f0 ff 00 |b:0 x:3 y:-1 w:0    <<==估计2个同时被按了.
I (1163319) log2file: 8 00 01 d0 ff 00 |b:0 x:1 y:-3 w:0
I (1163327) log2file: 7 00 02 d0 ff 00 |b:0 x:2 y:-3 w:0
```

- 获得录制记录时间 (20250917)

```
cli> alog mouse
I (670538) alog: logfile /eMMC/applog/mouse.log
cli> astop
I (709426) alog: logfile /eMMC/applog/mouse.log, 537 about 35 s
cli> atime mouse
I (723466) alog: logfile /eMMC/applog/mouse.log
I (723466) alog: #1, unitsize 5
I (724834) alog: finished rec: 537, 33752 ms
33 s, 752 ms
```

> 这个记录时就是35, 而实际时就是33s, 主要差别是,记录是在astop 时候认为结束. 而其实最后一条记录已经完成, 当中差了2s. 实际atime的时就就是准确的时间. 目前这个时间和真正播放 可能还是有差异的. 因为播放时候,做了8ms的规整. 所以精度就到秒, ms 这个级别应该是不准了.

- 直接控制鼠标(m10鼠标) ()
  看灯, 知道鼠标处于的通道, 下面的demo 鼠标接在#1 通道,

M10: x, y的范围 -2048~2047, wheel -128~127
button: 低5bit, 最低是 左按键.

--p: port #
--b: botton
--x: x
--y: y
--w: wheel
--v: verbose, display send buffer. 1: verbose 0 , default, not verbose.

m10 --port 1 --b xx --x xx --y xx --w xx

> 滚轮没测试过, 按键对应要根据adump的解析情况. 默认不给参数就是0.

```
cli> m10
m10: missing option --port=<p>
Command returned non-zero error code: 0x1 (ERROR)
cli> m10 --port 1  <<===所有都是默认值, 看不见效果
cli> m10 --port 1 --x 100  <<==连续发, 可以看到鼠标右移动
cli> m10 --port 1 --x 100
cli> m10 --port 1 --x -100 <<==连续发, 可以看到鼠标左移动
cli> m10 --port 1 --y 100  <<==连续发, 可以看到鼠标下移动
cli> m10 --port 1 --y 100
cli> m10 --port 1 --y -100 <<==连续发, 可以看到鼠标上移动
```

> !!!特别注意!!! 如果mouse replay 做过,直接做鼠标控制, 如果--port 1|2 发错了, 这样必须要reset 板子了. 因为不同的通道会设置对应的数据接收属性,这个只发生一次,如果发错了,就记录了, 所以必须reset.

v20250911以后, 增加日志解析和增加鼠标接口控制.

- 键盘通信编码说明

键盘的底层编码默认是8个字节

偏移 0	**1**	**2**	**3**	**4**	**5**	**6**	**7**
控制状态	reserv	key1	key2	key3	key4	key5	key6
byte	byte	byte	byte	byte	byte	byte	byte

按键状态

1. Bit 0: Left Control
2. Bit 1: Left Shift
3. Bit 2: Left Alt
4. Bit 3: Left GUI (Windows Key, Command Key)
5. Bit 4: Right Control
6. Bit 5: Right Shift
7. Bit 6: Right Alt (AltGr)
8. Bit 7: Right GUI

key 的编码不是ascii的编码，参考 https://gist.github.com/MightyPork/6da26e382a7ad91b5496ee55fdc73db2 的定义。

- adump 增加键盘编码解析 (没解析状态控制, 只是按可见字符输出,这样控制字符无法看到.)

下面是显示输入了大部分字符按键的记录.

键盘输入的内容:

```
abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ1234567890-=123!##$%^&*()_+
```

记录文件是 k-test

```
cli> adump --port 2 k-test
I (16898) alog: logfile /eMMC/applog/k-test.log
I (16986) log2file: numrec 85, recsize 12
I (16986) log2file: 0 00 00 04 00 00 00 00 00 |key: a
I (16986) log2file: 103 00 00 00 00 00 00 00 00
I (16986) log2file: 519 00 00 05 00 00 00 00 00 |key: b
I (17002) log2file: 95 00 00 00 00 00 00 00 00
I (17002) log2file: 280 00 00 06 00 00 00 00 00 |key: c
I (17002) log2file: 103 00 00 00 00 00 00 00 00
I (17010) log2file: 176 00 00 07 00 00 00 00 00 |key: d
I (17010) log2file: 87 00 00 00 00 00 00 00 00
I (17026) log2file: 271 00 00 08 00 00 00 00 00 |key: e
I (17026) log2file: 88 00 00 00 00 00 00 00 00
I (17026) log2file: 207 00 00 09 00 00 00 00 00 |key: f
I (17034) log2file: 88 00 00 00 00 00 00 00 00
I (17034) log2file: 215 00 00 0a 00 00 00 00 00 |key: g
I (17042) log2file: 95 00 00 00 00 00 00 00 00
I (17050) log2file: 335 00 00 0b 00 00 00 00 00 |key: h
I (17050) log2file: 111 00 00 00 00 00 00 00 00
I (17050) log2file: 144 00 00 0c 00 00 00 00 00 |key: i
I (17066) log2file: 120 00 00 00 00 00 00 00 00
I (17066) log2file: 183 00 00 0d 00 00 00 00 00 |key: j
I (17066) log2file: 95 00 00 00 00 00 00 00 00
I (17074) log2file: 359 00 00 0e 00 00 00 00 00 |key: k
I (17074) log2file: 95 00 00 00 00 00 00 00 00
I (17082) log2file: 328 00 00 0f 00 00 00 00 00 |key: l
I (17090) log2file: 111 00 00 00 00 00 00 00 00
I (17090) log2file: 191 00 00 10 00 00 00 00 00 |key: m
I (17106) log2file: 112 00 00 00 00 00 00 00 00
I (17106) log2file: 216 00 00 11 00 00 00 00 00 |key: n
I (17106) log2file: 95 00 00 00 00 00 00 00 00
I (17114) log2file: 400 00 00 12 00 00 00 00 00 |key: o
I (17114) log2file: 103 00 00 00 00 00 00 00 00
I (17114) log2file: 103 00 00 13 00 00 00 00 00 |key: p
I (17130) log2file: 112 00 00 00 00 00 00 00 00
I (17130) log2file: 320 00 00 14 00 00 00 00 00 |key: q
I (17130) log2file: 96 00 00 00 00 00 00 00 00
I (17138) log2file: 327 00 00 15 00 00 00 00 00 |key: r
I (17146) log2file: 79 00 00 00 00 00 00 00 00
I (17146) log2file: 135 00 00 16 00 00 00 00 00 |key: s
I (17154) log2file: 88 00 00 00 00 00 00 00 00
I (17154) log2file: 255 00 00 17 00 00 00 00 00 |key: t
I (17170) log2file: 80 00 00 00 00 00 00 00 00
I (17170) log2file: 1328 00 00 18 00 00 00 00 00 |key: u
I (17170) log2file: 95 00 00 00 00 00 00 00 00
I (17178) log2file: 191 00 00 19 00 00 00 00 00 |key: v
I (17178) log2file: 88 00 00 00 00 00 00 00 00
I (17178) log2file: 295 00 00 1a 00 00 00 00 00 |key: w
I (17194) log2file: 112 00 00 00 00 00 00 00 00
I (17194) log2file: 527 00 00 1b 00 00 00 00 00 |key: x
I (17194) log2file: 111 00 00 00 00 00 00 00 00
I (17210) log2file: 432 00 00 1c 00 00 00 00 00 |key: y
I (17210) log2file: 87 00 00 00 00 00 00 00 00
I (17210) log2file: 207 00 00 1d 00 00 00 00 00 |key: z
I (17218) log2file: 96 00 00 00 00 00 00 00 00
I (17218) log2file: 4519 00 00 39 00 00 00 00 00 |key:   <<==0x39 是 KEY_CAPSLOCK
```

```
I (17234) log2file: 122 00 00 00 00 00 00 00
I (17234) log2file: 1104 00 00 04 00 00 00 00 00 |key: a
I (17234) log2file: 95 00 00 00 00 00 00 00
I (17242) log2file: 183 00 00 05 00 00 00 00 00 |key: b
I (17242) log2file: 95 00 00 00 00 00 00 00
I (17250) log2file: 304 00 00 06 00 00 00 00 00 |key: c
I (17258) log2file: 96 00 00 00 00 00 00 00
I (17258) log2file: 183 00 00 07 00 00 00 00 00 |key: d
I (17258) log2file: 87 00 00 00 00 00 00 00
I (17274) log2file: 207 00 00 08 00 00 00 00 00 |key: e
I (17274) log2file: 88 00 00 00 00 00 00 00
I (17274) log2file: 208 00 00 09 00 00 00 00 00 |key: f
I (17282) log2file: 96 00 00 00 00 00 00 00
I (17282) log2file: 295 00 00 0a 00 00 00 00 00 |key: g
I (17298) log2file: 95 00 00 00 00 00 00 00
I (17298) log2file: 376 00 00 0b 00 00 00 00 00 |key: h
I (17298) log2file: 95 00 00 00 00 00 00 00
I (17306) log2file: 120 00 00 0c 00 00 00 00 00 |key: i
I (17314) log2file: 95 00 00 00 00 00 00 00
I (17314) log2file: 407 00 00 0d 00 00 00 00 00 |key: j
I (17322) log2file: 95 00 00 00 00 00 00 00
I (17322) log2file: 136 00 00 0e 00 00 00 00 00 |key: k
I (17322) log2file: 103 00 00 00 00 00 00 00
I (17338) log2file: 560 00 00 0f 00 00 00 00 00 |key: l
I (17338) log2file: 120 00 00 00 00 00 00 00
I (17338) log2file: 111 00 00 10 00 00 00 00 00 |key: m
I (17346) log2file: 95 00 00 00 00 00 00 00
I (17346) log2file: 232 00 00 11 00 00 00 00 00 |key: n
I (17362) log2file: 88 00 00 00 00 00 00 00
I (17362) log2file: 368 00 00 12 00 00 00 00 00 |key: o
I (17362) log2file: 87 00 00 00 00 00 00 00
I (17378) log2file: 111 00 00 13 00 00 00 00 00 |key: p
I (17490) log2file: numrec 84, recsize 12
I (17490) log2file: 0 00 00 00 00 00 00 00
I (17490) log2file: 848 00 00 14 00 00 00 00 00 |key: q
I (17490) log2file: 63 00 00 00 00 00 00 00
I (17506) log2file: 544 00 00 15 00 00 00 00 00 |key: r
I (17506) log2file: 79 00 00 00 00 00 00 00
I (17506) log2file: 136 00 00 16 00 00 00 00 00 |key: s
I (17514) log2file: 87 00 00 00 00 00 00 00
I (17514) log2file: 176 00 00 17 00 00 00 00 00 |key: t
I (17530) log2file: 87 00 00 00 00 00 00 00
I (17530) log2file: 424 00 00 18 00 00 00 00 00 |key: u
I (17530) log2file: 63 00 00 00 00 00 00 00
I (17538) log2file: 231 00 00 19 00 00 00 00 00 |key: v
I (17546) log2file: 96 00 00 00 00 00 00 00
I (17546) log2file: 351 00 00 1a 00 00 00 00 00 |key: w
I (17554) log2file: 88 00 00 00 00 00 00 00
I (17554) log2file: 768 00 00 1b 00 00 00 00 00 |key: x
I (17554) log2file: 111 00 00 00 00 00 00 00
I (17570) log2file: 280 00 00 1c 00 00 00 00 00 |key: y
I (17570) log2file: 87 00 00 00 00 00 00 00
I (17570) log2file: 159 00 00 1d 00 00 00 00 00 |key: z
I (17578) log2file: 87 00 00 00 00 00 00 00
I (17586) log2file: 1368 00 00 39 00 00 00 00 00 |key:
I (17586) log2file: 79 00 00 00 00 00 00 00
I (17594) log2file: 1307 00 00 1e 00 00 00 00 00 |key: 1
I (17594) log2file: 87 00 00 00 00 00 00 00
I (17610) log2file: 271 00 00 1f 00 00 00 00 00 |key: 2
I (17610) log2file: 120 00 00 00 00 00 00 00
I (17610) log2file: 511 00 00 20 00 00 00 00 00 |key: 3
I (17618) log2file: 143 00 00 00 00 00 00 00
I (17618) log2file: 407 00 00 21 00 00 00 00 00 |key: 4
I (17618) log2file: 120 00 00 00 00 00 00 00
I (17634) log2file: 240 00 00 22 00 00 00 00 00 |key: 5
I (17634) log2file: 103 00 00 00 00 00 00 00
I (17634) log2file: 208 00 00 23 00 00 00 00 00 |key: 6
I (17642) log2file: 71 00 00 00 00 00 00 00
I (17650) log2file: 256 00 00 24 00 00 00 00 00 |key: 7
I (17650) log2file: 79 00 00 00 00 00 00 00
I (17658) log2file: 256 00 00 25 00 00 00 00 00 |key: 8
I (17658) log2file: 79 00 00 00 00 00 00 00
I (17674) log2file: 247 00 00 26 00 00 00 00 00 |key: 9
I (17674) log2file: 87 00 00 00 00 00 00 00
```

```
I (17674) log2file: 256 00 00 27 00 00 00 00 00 |key: 0
I (17682) log2file: 103 00 00 00 00 00 00 00 00
I (17682) log2file: 1800 00 00 2d 00 00 00 00 00 |key: -
I (17690) log2file: 71 00 00 00 00 00 00 00 00
I (17698) log2file: 392 00 00 2e 00 00 00 00 00 |key: =
I (17698) log2file: 87 00 00 00 00 00 00 00 00
I (17698) log2file: 5432 00 00 39 00 00 00 00 00 |key:
I (17714) log2file: 130 00 00 00 00 00 00 00 00
I (17714) log2file: 1216 00 00 1e 00 00 00 00 00 |key: 1
I (17714) log2file: 103 00 00 00 00 00 00 00 00
I (17722) log2file: 231 00 00 1f 00 00 00 00 00 |key: 2
I (17722) log2file: 128 00 00 00 00 00 00 00 00
I (17738) log2file: 335 00 00 20 00 00 00 00 00 |key: 3
I (17738) log2file: 127 00 00 00 00 00 00 00 00
I (17738) log2file: 9519 00 00 39 00 00 00 00 00 |key:
I (17746) log2file: 143 00 00 00 00 00 00 00 00
I (17754) log2file: 2516 02 00 00 00 00 00 00 00            <<===02 是左边shift
I (17754) log2file: 1311 02 00 1e 00 00 00 00 00 |key: 1    <<== 产生!
I (17762) log2file: 119 02 00 00 00 00 00 00 00
I (17762) log2file: 360 02 00 20 00 00 00 00 00 |key: 3
I (17762) log2file: 111 02 00 00 00 00 00 00 00
I (17778) log2file: 1071 02 00 20 00 00 00 00 00 |key: 3
I (17778) log2file: 63 02 00 00 00 00 00 00 00
I (17778) log2file: 207 02 00 21 00 00 00 00 00 |key: 4
I (17786) log2file: 88 02 00 00 00 00 00 00 00
I (17786) log2file: 295 02 00 22 00 00 00 00 00 |key: 5
I (17802) log2file: 95 02 00 00 00 00 00 00 00
I (17802) log2file: 216 02 00 23 00 00 00 00 00 |key: 6
I (17802) log2file: 88 02 00 00 00 00 00 00 00
I (17810) log2file: 295 02 00 24 00 00 00 00 00 |key: 7
I (17818) log2file: 87 02 00 00 00 00 00 00 00
I (17818) log2file: 215 02 00 25 00 00 00 00 00 |key: 8
I (17826) log2file: 80 02 00 00 00 00 00 00 00
I (17826) log2file: 240 02 00 26 00 00 00 00 00 |key: 9
I (17826) log2file: 87 02 00 00 00 00 00 00 00
I (17842) log2file: 304 02 00 27 00 00 00 00 00 |key: 0
I (17842) log2file: 80 02 00 00 00 00 00 00 00
I (17842) log2file: 335 02 00 2d 00 00 00 00 00 |key: -
I (17850) log2file: 79 02 00 00 00 00 00 00 00
I (17858) log2file: 608 02 00 2e 00 00 00 00 00 |key: =
I (17866) log2file: 79 02 00 00 00 00 00 00 00
I (17866) log2file: 2728 00 00 00 00 00 00 00 00
I (17866) log2file: 8 00 00 00 00 00 00 00 00
I (17882) log2file: numrec 0, recsize 12
```

这里要注意,部分的输入是通过状态来处理的. 特别注意每个按键后, 一般有全0表示release

- 直接控制键盘 kpad

参数就是用来构成这样的8个byte .
--port 1 | 2
--s 对应 buff[0]， 这个是对应控制按键：shift, ctl gui 这种。根据bit 做或操作。
--x1 对应 buff[2]
--x2 对应 buff[3]
--x3 对应 buff[4]
--x4 对应 buff[5]
--x5 对应 buff[6]
--x6 对应 buff[7]
--rel 自动release keypad, 默认是自动release. 1: 16ms release, <16ms 是不合理的， 按键没有那么快弹起。如果设置500ms, 就是常按后，500ms 释放
--d 延迟, 默认5s , 0 不延迟
--v verbose , 输出发送的缓冲内容, 用来检查.

主要使用方式, status, xN 直接映射buffer, 比较直接. --rel 0 控制接口不release 按键,这样导致一个按键处于长按状态, 目前release 是整个key, status 都释放的. 默认 --rel 1, 程序自动释放按键. 这样只要一条命令,就可以完成输入. --d 是延迟参数,按键什么时候发

送, 在console 测试时候, 如果没有延迟参数,因为cli的串口原因,可能会无法看到输入.(太快了), 如果在延迟的情况下, 切换输入窗口, 就可以看到输入. 正常人输入最多同时产生2个输入的key, 但是软件接口,可以一次送6个输入key.

```
cli> kpad --port 2 --x1 0x11 --x2 0x12 --v 1   <<--默认延迟5s, 切换到可输入的软件,按键是 no
delay 5 start input
I (2145690) log2file: 00 00 11 12 00 00 00 00
I (2145706) log2file: 00 00 00 00 00 00 00 00
cli> kpad --port 2 --x1 0x11 --v 1             <<---默认延迟5s, 切换到可输入的软件,按键是 n
delay 5 start input
I (2160554) log2file: 00 00 11 00 00 00 00 00
I (2160570) log2file: 00 00 00 00 00 00 00 00
```

> !!!特别注意!!! 如果mouse replay 做过,直接做鼠标控制, 如果--port 1|2 发错了, 这样必须要reset 板子了. 因为不同的通道会设置对应的数据接收属性,这个只发生一次,如果发错了,就记录了, 所以必须reset.

API 集成方式

在手工能控制的情况下, 用API 就是实现这个串口的CLI. 用golang 的串口的包,打开串口, 然后进行收发数据,解析这个串口输出的i/o就可以了. 命令简单, 相对不复杂.

已经知道的BUG

第一次回放,可能会遇到窗口飘了. (还是另外一个功能没有完全剥离,导致要初始设置失败.) 后面回放就会正常. 这个以后再解决. 最简单的方式就是, 回放一个垃圾小片段.

```
Author: minye.li
Init: 2025/06/23
Updated: 2025/09/12
```